

**KHOA KỸ THUẬT – CÔNG NGHỆ**



Chương 6

# KỸ THUẬT LẬP TRÌNH

GV biên soạn: Trần Kim Hương

[tkhuong@dthu.edu.vn](mailto:tkhuong@dthu.edu.vn)

Software Engineering



# NỘI DUNG

## KỸ THUẬT LẬP TRÌNH

**KHÁI NIỆM KỸ THUẬT LẬP TRÌNH**

**PHƯƠNG PHÁP LẬP TRÌNH**

**NGÔN NGỮ LẬP TRÌNH**

**PHONG CÁCH LẬP TRÌNH**

**KỸ THUẬT LẬP TRÌNH**



# 1. KHÁI NIỆM LẬP TRÌNH HIỆU QUẢ

## ❖ Sản phẩm phần mềm tốt khi?

- Phân tích tốt
- Thiết kế tốt
- Lập trình tốt
- Kiểm thử chặt chẽ

## ❖ Kỹ thuật lập trình tốt?

- Chuyên nghiệp (tuân theo các chuẩn)
- Ổn định
- Hiệu quả



# 1. KHÁI NIỆM LẬP TRÌNH HIỆU QUẢ

## ❖ Lập trình hiệu quả:

- Tốc độ phát triển cao hơn
  - Năng lực biểu diễn cao hơn
  - Khả năng sử dụng lại cao hơn
- Dễ bảo trì hơn
  - Dễ hiểu, dễ sửa lỗi, thích nghi
- Chất lượng cao hơn
  - Sử dụng các cấu trúc an toàn hơn

→ Chương trình cần dễ hiểu



# NỘI DUNG

## KỸ THUẬT LẬP TRÌNH

KHÁI NIỆM KỸ THUẬT LẬP TRÌNH

**PHƯƠNG PHÁP LẬP TRÌNH**

NGÔN NGỮ LẬP TRÌNH

PHONG CÁCH LẬP TRÌNH

KỸ THUẬT LẬP TRÌNH



## 2. PHƯƠNG PHÁP LẬP TRÌNH

- ❖ Lập trình tuần tự (tuyến tính)
- ❖ Lập trình có cấu trúc (thủ tục)
- ❖ Lập trình hướng chức năng
- ❖ Lập trình hướng đối tượng
- ❖ Lập trình logic (kỹ thuật thể hệ thứ 4)



## 2.1 Lập trình tuần tự

- ❖ Không có/ thiếu các lệnh có cấu trúc (**for**, **while**, **do while**)
- ❖ Lạm dụng các lệnh **GOTO**
- ❖ Thiếu khả năng khai báo biến cục bộ
  - độ ghép nối cao
  - Chương trình khó hiểu, khó sửa, dễ sinh lỗi
  - Dùng ngôn ngữ thế hệ 1, 2 (assembly, basic,...)



## 2.2 Lập trình có cấu trúc

- ❖ Sử dụng các lệnh có cấu trúc (for, while, do while)
  - ❖ Hạn chế/ cấm dùng GOTO
  - ❖ Sử dụng chương trình con, biến cục bộ
- Dễ hiểu hơn, an toàn hơn
- Ngôn ngữ dùng thể hệ 2, 3: Fortran, pascal, C,...





## 2.3 Lập trình hướng chức năng

- ❖ Dựa trên nguyên tắc **ghép nối dữ liệu**
  - Trao đổi dữ liệu bằng tham số và giá trị trả lại
  - Loại bỏ hoàn toàn dữ liệu dùng chung
- ❖ Loại bỏ các hiệu ứng phụ khi sửa đổi các module chương trình; nâng cao tính tái sử dụng
- ❖ *Ví dụ*: LISP



## 2.4 Lập trình hướng đối tượng

- ❖ Che giấu thông tin
- ❖ Thao tác với dữ liệu qua các giao diện xác định
- ❖ Kế thừa
  - Cục bộ hơn
  - Dễ tái sử dụng hơn
  - Thuận tiện cho các ứng dụng lớn
- Ngôn ngữ lập trình hướng đối tượng: C++, Java, C#,...



## 2.5 Lập trình logic

- ❖ Tách tri thức về bài toán khỏi kỹ thuật lập trình
- ❖ Mô tả tri thức
  - Các quy tắc
  - Các sự kiện
  - Mục tiêu
- ❖ Hệ thống tự chứng minh
  - Tìm đường đi đến mục tiêu
- ❖ *Ví dụ*: Prolog



# NỘI DUNG

## KỸ THUẬT LẬP TRÌNH

KHÁI NIỆM KỸ THUẬT LẬP TRÌNH

PHƯƠNG PHÁP LẬP TRÌNH

**NGÔN NGỮ LẬP TRÌNH**

PHONG CÁCH LẬP TRÌNH

KỸ THUẬT LẬP TRÌNH



# 3. NGÔN NGỮ LẬP TRÌNH

## ❖ Lựa chọn ngôn ngữ dựa vào:

- Đặc trưng của ngôn ngữ
  - Năng lực (kiểu biến, các cấu trúc)
  - Tính khả chuyển
  - Mức độ hỗ trợ của các ngôn ngữ
- Miền ứng dụng của ngôn ngữ
  - Lập trình hệ thống
  - Nghiệp vụ, kinh doanh
  - Khoa học kỹ thuật
  - Trí tuệ nhân tạo
- Năng lực, kinh nghiệm của nhóm phát triển
- Yêu cầu của khách hàng



## 3.1 Đặc trưng của ngôn ngữ

### ❖ Năng lực của ngôn ngữ

#### ▪ Ngôn ngữ bậc cao:

- Có cấu trúc, câu lệnh phong phú
- Hỗ trợ nhiều kiểu dữ liệu
- Hỗ trợ con trỏ, đệ quy
- Hỗ trợ hướng đối tượng
- Thư viện phong phú

→ **Nên** dùng ngôn ngữ bậc cao



## 3.1 Đặc trưng của ngôn ngữ

### ❖ Tính khả chuyển

- Là yếu tố quan trọng của ngôn ngữ, cần khi
    - Thay đổi phần cứng
    - Thay đổi OS
  - Java khả chuyển
  - Các ngôn ngữ thông dịch (script) khả chuyển
- sử dụng các tính năng chuẩn của ngôn ngữ
- Sử dụng script khi có thể



## 3.1 Đặc trưng của ngôn ngữ

### ❖ Công cụ hỗ trợ của ngôn ngữ

#### ▪ Trình biên dịch hiệu quả

- Biên dịch tốc độ cao
- Khả năng tối ưu cao
- Khai thác các tập lệnh, kiến trúc phần cứng mới

#### ▪ Các công cụ trợ giúp hiệu quả

- Editor, debugger,...
- IDE (Integrated Develop Environment)
- Môi trường UNIX thường không dùng IDE





## 3.2 Miền ứng dụng và ngôn ngữ

### ❖ Phần mềm hệ thống

- Hiệu quả, dễ mở rộng
- C, C++

### ❖ Hệ thời gian thực: C, C++, Ada, Assembly

### ❖ Phần mềm nhúng: C++, Java

### ❖ Phần mềm khoa học kỹ thuật:

- Tính toán chính xác, thư viện toán học mạnh, dễ dàng song song hóa
- FORTRAN vẫn phổ biến



## 3.2 Miền ứng dụng và ngôn ngữ

### ❖ Phần mềm nghiệp vụ:

- CSDL: Oracle, DB2, SQL Server, MySQL,...
- Ngôn ngữ: Foxpro, COBOL, VB.NET, VC++, ...

### ❖ Trí tuệ nhân tạo

- Lisp, Prolog, OPS5,...

### ❖ Lập trình Web/ CGI:

- Perl, ASP, PHP, Java, JavaScript, ...



# NỘI DUNG

## KỸ THUẬT LẬP TRÌNH

KHÁI NIỆM KỸ THUẬT LẬP TRÌNH

PHƯƠNG PHÁP LẬP TRÌNH

NGÔN NGỮ LẬP TRÌNH

**PHONG CÁCH LẬP TRÌNH**

KỸ THUẬT LẬP TRÌNH



# 4. PHONG CÁCH LẬP TRÌNH

❖ Gồm các yếu tố:

- Cách đặt tên hàm và biến
- Cách xây dựng câu lệnh, cấu trúc chương trình
- Cách viết chú thích

→ Hướng đến phong cách làm cho mã nguồn

→ Dễ hiểu, dễ sửa đổi

→ An toàn (ít lỗi)

***Người khác có thể hiểu được, bảo trì được!!!***



# Chú thích

## ❖ Mọi điều được chú thích trong chương trình

- Mục đích sử dụng của các biến
- Chức năng của khối lệnh, câu lệnh
  - Các lệnh điều khiển
  - Các lệnh phức tạp
- Chú thích các module
  - Mục đích, chức năng của module
  - Tham số, giá trị trả lại (giao diện)
  - Các module thuộc cấp
  - Cấu trúc, thuật toán
  - Nhiệm vụ của các biến cục bộ
  - Tác giả, người kiểm tra, thời gian



# Đặt tên

- ❖ Đặt tên biến, tên hàm có nghĩa, gợi nhớ
  - Sử dụng các ký hiệu, từ tiếng Anh có nghĩa
  - Làm cho dễ đọc
    - *Vd*: dùng DateOfBirth hoặc date\_of\_birth  
→ Không viết dateofbirth
  - Tránh đặt tên quá dài
  - Thống nhất cách dùng
    - *Vd*: i cho vòng lặp, tm cho các giá trị tạm...



# Câu lệnh

- ❖ Các câu lệnh phải mô tả cấu trúc
  - Tụt lề, dễ đọc, dễ hiểu
- ❖ Làm đơn giản các lệnh
  - Mỗi lệnh trên một dòng
  - Triển khai các biểu thức phức tạp
  - Hạn chế truyền tham số là kết quả của hàm, biểu thức

Vd: `printf ("%s", strcpy(des, src));`

- Tránh các cấu trúc phức tạp
  - Các lệnh if lồng nhau
  - Điều khiển phủ định if not



# Hàm và biến cục bộ

- ❖ Chương trình cần được chia thành nhiều module (hàm)
- ❖ Không viết hàm quá dài
  - Không quá 2 trang màn hình
- ❖ Không dùng quá nhiều biến cục bộ
  - Không thể theo dõi đồng thời hoạt động của nhiều biến





# Xử lý lỗi

## ❖ Có thể phát hiện lỗi trong khi thực hiện

- Lỗi chia 0
- Lỗi input/ output

## ❖ Xử lý lỗi

- Nhất quán trong xử lý: phân loại lỗi, thống nhất định dạng thông báo
- Phân biệt output và thông báo lỗi



# Phong cách lập trình tốt

- ❖ Tuân theo các **chuẩn** thông dụng
- ❖ Chuẩn được chấp nhận rộng rãi hơn, dễ hiểu hơn
- ❖ Chú giải đầy đủ mỗi khi không tuân theo chuẩn

***“Người khác có thể hiểu được không?”***



# NỘI DUNG

## KỸ THUẬT LẬP TRÌNH

KHÁI NIỆM KỸ THUẬT LẬP TRÌNH

PHƯƠNG PHÁP LẬP TRÌNH

NGÔN NGỮ LẬP TRÌNH

PHONG CÁCH LẬP TRÌNH

**KỸ THUẬT LẬP TRÌNH**



# 5. KỸ THUẬT LẬP TRÌNH

❖ kỹ thuật lập trình tốt dựa trên các yếu tố:

- Lập trình có cấu trúc

- Dùng các lệnh có cấu trúc
- Module hóa
- Hạn chế dùng các cấu trúc nguy hiểm

- Đóng gói/ che giấu thông tin

- Xây dựng kiểu dữ liệu trừu tượng
- Hạn chế thao tác trực tiếp lên thuộc tính



# Tránh các cấu trúc nguy hiểm

## ❖ Số thực:

- Các phép toán làm tròn, kết quả không chính xác tuyệt đối
- So sánh (=) hai số thực là không khả thi

## ❖ Con trỏ

- Có khả năng gây lỗi nghiêm trọng
- Dễ nhầm

Vd: double r;

int\* n = &r;



# Tránh các cấu trúc nguy hiểm

## ❖ Cấp phát bộ nhớ động

- Quên cấp phát
- Quên giải phóng

## ❖ Độ quy

- Khó hiểu
- Dễ nhầm điều kiện dừng



# Lập trình phòng thủ

## ❖ Nhiều lệnh có khả năng sinh lỗi

- Lệnh vào/ ra
- Các phép toán
- Thao tác với bộ nhớ
- Truyền tham số sai kiểu

## ❖ Dự đoán khả năng xuất hiện lỗi

## ❖ Khắc phục lỗi

- Lưu trạng thái an toàn
- Quay lại trạng thái an toàn gần nhất



# Lập trình phòng thủ

## ❖ Lệnh vào/ ra

- Dữ liệu vào không hợp lệ
- Tràn bộ nhớ (kiểu ký tự)
- Lỗi thao tác file (sai tên, chưa được mở,...)

## ❖ Các phép toán

- Lỗi chia 0
- Tràn số
- So sánh số thực (bằng nhau)



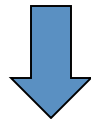


# Lập trình phòng thủ

❖ Ví dụ:

```
FILE* fp;
```

```
fp = fopen ("data","r");
```



```
FILE* fp;
```

```
If (NULL == (fp=fopen("data", "r"))) {  
    fprintf (stderr, "can not open file...");  
    .....
```

```
}
```



# Hướng hiệu quả thực hiện

- ❖ Phần mềm ngày càng phức tạp, đa dạng
  - Mô phỏng
  - Ứng dụng thời gian thực
  - Phần mềm nhúng
  - Trò chơi
- ❖ Hiệu quả thực hiện luôn cần được xem xét
  - Thuật toán hiệu quả
  - Kỹ thuật lập trình hiệu quả
  - Ngôn ngữ lập trình hiệu quả



# Câu hỏi

1. Kỹ thuật lập trình tốt thể hiện ở chỗ nào? Hệ quả của nó?
2. Nêu các phương pháp lập trình đã có? Đặc trưng của mỗi loại là gì?
3. Tiêu chuẩn lựa chọn ngôn ngữ lập trình?
4. Thế nào là ngôn ngữ khả chuyển? Cho ví dụ?
5. Nêu các miền ứng dụng và ngôn ngữ thích hợp với nó?
6. Các yếu tố tạo ra phong cách lập trình là gì? Nó hướng tới phần mềm có đặc trưng gì?



# Câu hỏi

7. Giải thích cách làm: chú thích? Đặt tên? Viết câu lệnh?..
8. Tiêu chuẩn cho phong cách lập trình tốt?



# Câu hỏi thảo luận

